1. fcfs

```
#include <stdio.h>

int main() {
    int n, i, j;
    int at[20], bt[20], wt[20], tat[20], ct[20], rt[20], p[20];
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("\nProcess P%d\n", p[i]);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
    }

    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(at[i] > at[j]) {
                int temp;
                temp = at[i];
                at[i] = at[j];
                at[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    ct[0] = at[0] + bt[0];
    wt[0] = 0;
    rt[0] = 0;
    tat[0] = ct[0] - at[0];

    for(i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];

        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        rt[i] = wt[i];
    }

    for(i = 0; i < n; i++) {
        avg_wt += wt[i];
        avg_tat += tat[i];
        avg_rt += rt[i];
```

```
    }

    printf("\n-------------------------------------------------------------------\n");
    printf("Process\tAT\tBT\tCT\tWT\tTAT\tRT\n");
    printf("-------------------------------------------------------------------\n");

    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i], ct[i], wt[i], tat[i], rt[i]);
    }

    printf("-------------------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avg_wt / n);
    printf("Average Turnaround Time = %.2f\n", avg_tat / n);
    printf("Average Response Time = %.2f\n", avg_rt / n);

    return 0;
}
```

2. sjf

```
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int id;
    int at;
    int bt;
    int ct;
    int wt;
    int tat;
    int rt;
    bool is_completed;
};

int main() {
    int n;

    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;

    struct Process p[n];


    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("\nProcess P%d\n", p[i].id);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        p[i].is_completed = false;
    }

    int completed = 0;
```

```
int current_time = 0;
float total_wt = 0, total_tat = 0, total_rt = 0;

struct Process output[n];
int out_index = 0;


while (completed != n) {
    int idx = -1;
    int min_bt = 1e9;

    for (int i = 0; i < n; i++) {
        if (p[i].at <= current_time && !p[i].is_completed) {
            if (p[i].bt < min_bt) {
                min_bt = p[i].bt;
                idx = i;
            }
            else if (p[i].bt == min_bt) {
                if (p[i].at < p[idx].at) {
                    idx = i;
                }
            }
        }
    }

    if (idx != -1) {
        p[idx].rt = current_time - p[idx].at;
        p[idx].ct = current_time + p[idx].bt;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;

        total_wt += p[idx].wt;
        total_tat += p[idx].tat;
        total_rt += p[idx].rt;

        p[idx].is_completed = true;
        completed++;
        current_time = p[idx].ct;

        output[out_index++] = p[idx];
    } else {
        current_time++;
    }
}


printf("\n_____\n");
printf("Process AT\tBT\tCT\tWT\tTAT\tRT\n");
printf("_____\n");

for (int i = 0; i < n; i++) {
```

```
            printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                output[i].id, output[i].at, output[i].bt,
                output[i].ct, output[i].wt, output[i].tat, output[i].rt);
        }
        printf("_____\n");

        printf("Average Waiting Time = %.2f\n", total_wt / n);
        printf("Average Turnaround Time = %.2f\n", total_tat / n);
        printf("Average Response Time = %.2f\n", total_rt / n);

        return 0;
    }
```

3. priority

```
#include <stdio.h>

int main() {
    int n, i, completed = 0, time = 0;
    int at[20], bt[20], pr[20], wt[20], tat[20], rt[20], ct[20], done[20] = {0}, p[20];
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("\nProcess P%d\n", p[i]);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority: ");
        scanf("%d", &pr[i]);
    }

    while (completed < n) {
        int idx = -1;
        int highest = 9999;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && done[i] == 0) {
                if (pr[i] < highest) {
                    highest = pr[i];
                    idx = i;
                }
                if (pr[i] == highest) {
```

```
                if (at[i] < at[idx]) {
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            time++;
        } else {
            rt[idx] = time - at[idx];
            ct[idx] = time + bt[idx];
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            avg_wt += wt[idx];
            avg_tat += tat[idx];
            avg_rt += rt[idx];

            time = ct[idx];
            done[idx] = 1;
            completed++;
        }
    }

    printf("\nProcess\tAT\tBT\tPriority\tWT\tTAT\tRT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\n", p[i], at[i], bt[i], pr[i], wt[i], tat[i], rt[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time = %.2f", avg_tat / n);
    printf("\nAverage Response Time = %.2f\n", avg_rt / n);

    return 0;
}
```

4. Round robin

```
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int id;
    int at;
    int bt;
```

```
    int rem_bt;
    int ct;
    int wt;
    int tat;
    int rt;
    bool is_started;
};

int main() {
    int n, quantum;

    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;

    struct Process p[n];

    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("\nProcess P%d\n", p[i].id);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        p[i].rem_bt = p[i].bt;
        p[i].is_started = false;
    }

    printf("\nEnter Time Quantum: ");
    scanf("%d", &quantum);

    int completed = 0;
    int current_time = 0;
    float total_wt = 0, total_tat = 0, total_rt = 0;

    struct Process output[n];
    int out_index = 0;

    while (completed != n) {
        bool idle = true;

        for (int i = 0; i < n; i++) {
            if (p[i].at <= current_time && p[i].rem_bt > 0) {
                idle = false;

                if (!p[i].is_started) {
                    p[i].rt = current_time - p[i].at;
                    p[i].is_started = true;
                }
```

```
            if (p[i].rem_bt > quantum) {
                current_time += quantum;
                p[i].rem_bt -= quantum;
            }
            else {
                current_time += p[i].rem_bt;
                p[i].rem_bt = 0;
                p[i].ct = current_time;
                p[i].tat = p[i].ct - p[i].at;
                p[i].wt = p[i].tat - p[i].bt;

                total_wt += p[i].wt;
                total_tat += p[i].tat;
                total_rt += p[i].rt;

                completed++;

                output[out_index++] = p[i];
            }
        }
    }

    if (idle) {
        current_time++;
    }
}

printf("\n_____\n");
printf("Process\tAT\tBT\tCT\tWT\tTAT\tRT\n");
printf("_____\n");

for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           output[i].id, output[i].at, output[i].bt,
           output[i].ct, output[i].wt, output[i].tat, output[i].rt);
}
printf("_____\n");

printf("Average Waiting Time = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);
printf("Average Response Time = %.2f\n", total_rt / n);

return 0;
}
```

5. Producer/Consumer Problem

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5
int buffer[SIZE];
int in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;

int num_produced, num_consumed;

void *producer(void *arg) {
    int item;
    int count = 0;
    while (count < num_produced) {
        item = rand() % 100;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
        count++;
    }
    return NULL;
}

void *consumer(void *arg) {
    int item;
    int count = 0;
    while (count < num_consumed) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&empty);
        sleep(1);
        count++;
    }
    return NULL;
}

int main() {
    printf("Enter number of items to produce: ");
    if (scanf("%d", &num_produced) != 1 || num_produced <= 0) return 1;

    printf("Enter number of items to consume: ");
    if (scanf("%d", &num_consumed) != 1 || num_consumed <= 0) return 1;

    pthread_t p, c;
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

6. Bounded buffer

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
int *buffer;
int size;
int in = 0, out = 0;

sem_t empty, full, mutex;

void *producer(void *arg)
{
    int id = *(int *)arg;
    free(arg);

    int item = rand() % 100;

    sem_wait(&empty);
    sem_wait(&mutex);

    buffer[in] = item;
    printf("Producer %d produced %d at position %d\n", id, item, in);
    in = (in + 1) % size;

    sem_post(&mutex);
    sem_post(&full);

    return NULL;
}

void *consumer(void *arg)
{
    int id = *(int *)arg;
    free(arg);

    sem_wait(&full);
    sem_wait(&mutex);

    int item = buffer[out];
    printf("Consumer %d consumed %d from position %d\n", id, item, out);
    out = (out + 1) % size;

    sem_post(&mutex);
    sem_post(&empty);

    return NULL;
```

```
}

int main()
{
    int producers, consumers;

    printf("Enter Buffer Size: ");
    scanf("%d", &size);

    printf("Enter Number of Producers: ");
    scanf("%d", &producers);

    printf("Enter Number of Consumers: ");
    scanf("%d", &consumers);

    buffer = (int *)malloc(size * sizeof(int));

    pthread_t p[producers], c[consumers];

    sem_init(&empty, 0, size);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    // Create Producer Threads
    for (int i = 0; i < producers; i++)
    {
        int *id = (int *)malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&p[i], NULL, producer, id);
    }

    // Create Consumer Threads
    for (int i = 0; i < consumers; i++)
    {
        int *id = (int *)malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&c[i], NULL, consumer, id);
    }

    // Wait for Producers
    for (int i = 0; i < producers; i++)
        pthread_join(p[i], NULL);
```

```
    // Wait for Consumers
    for (int i = 0; i < consumers; i++)
        pthread_join(c[i], NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    free(buffer);

    printf("\nBounded Buffer Problem Completed Successfully.\n");

    return 0;
}
```

7.reader writers problem

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t wrt;
pthread_mutex_t mutex;
int rcnt = 0;
int shared_data = 0;

void *reader(void *rno) {
    pthread_mutex_lock(&mutex);
    rcnt++;
    if(rcnt == 1) {
        sem_wait(&wrt);
    }
    pthread_mutex_unlock(&mutex);

    printf("Reader %d: read shared_data as %d\n", *((int *)rno), shared_data);

    pthread_mutex_lock(&mutex);
    rcnt--;
    if(rcnt == 0) {
        sem_post(&wrt);
    }
    pthread_mutex_unlock(&mutex);
    return NULL;
```

```
}

void *writer(void *wno) {
    sem_wait(&wrt);
    shared_data += 5;
    printf("Writer %d: write shared_data to %d\n", *((int *)wno), shared_data);
    sem_post(&wrt);
    return NULL;
}

int main() {
    int num_readers, num_writers;

    printf("Enter number of readers: ");
    if (scanf("%d", &num_readers) != 1 || num_readers <= 0) return 1;

    printf("Enter number of writers: ");
    if (scanf("%d", &num_writers) != 1 || num_writers <= 0) return 1;

    pthread_t r[num_readers], w[num_writers];
    int r_id[num_readers], w_id[num_writers];

    pthread_mutex_init(&mutex, NULL);
    sem_init(&wrt, 0, 1);

    int i = 0, j = 0;
    while (i < num_readers || j < num_writers) {
        if (i < num_readers) {
            r_id[i] = i + 1;
            pthread_create(&r[i], NULL, reader, &r_id[i]);
            i++;
        }
        if (j < num_writers) {
            w_id[j] = j + 1;
            pthread_create(&w[j], NULL, writer, &w_id[j]);
            j++;
        }
    }

    for (i = 0; i < num_readers; i++) pthread_join(r[i], NULL);
    for (j = 0; j < num_writers; j++) pthread_join(w[j], NULL);

    pthread_mutex_destroy(&mutex);
    sem_destroy(&wrt);
    return 0;
}
```

8.dining philosophers

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

int n;
pthread_mutex_t *forks;

void *philosopher(void *arg)
{
    int id = *(int *)arg;

    int left = id;
    int right = (id + 1) % n;

    printf("Philosopher %d is Thinking\n", id);

    // Deadlock Prevention
    if (id % 2 == 0)
    {
        pthread_mutex_lock(&forks[left]);
        pthread_mutex_lock(&forks[right]);
    }
    else
    {
        pthread_mutex_lock(&forks[right]);
        pthread_mutex_lock(&forks[left]);
    }

    printf("Philosopher %d is Eating\n", id);

    sleep(1);

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d Finished Eating\n", id);

    return NULL;
}

int main()
{
    printf("Enter Number of Philosophers: ");
```

```
    scanf("%d", &n);

    pthread_t philosophers[n];
    int id[n];

    forks = (pthread_mutex_t *)malloc(n * sizeof(pthread_mutex_t));

    for (int i = 0; i < n; i++)
        pthread_mutex_init(&forks[i], NULL);

    for (int i = 0; i < n; i++)
    {
        id[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < n; i++)
        pthread_join(philosophers[i], NULL);

    for (int i = 0; i < n; i++)
        pthread_mutex_destroy(&forks[i]);

    free(forks);

    printf("\nDining Philosophers Problem Completed Successfully.\n");

    return 0;
}
```

9. Sleeping berber

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>

sem_t customers;
sem_t barber;
pthread_mutex_t mutex;

int waiting = 0;
int chairs;
```

```
void *barber_func(void *arg)
{
   while (1)
   {
      sem_wait(&customers);

      pthread_mutex_lock(&mutex);

      waiting--;

      printf("\nBarber is cutting hair...");
      printf("\nWaiting customers = %d\n", waiting);

      sem_post(&barber);

      pthread_mutex_unlock(&mutex);

      sleep(2);
   }
}

void *customer_func(void *num)
{
   int id = *(int *)num;

   pthread_mutex_lock(&mutex);

   if (waiting < chairs)
   {
      waiting++;

      printf("\nCustomer %d entered shop", id);
      printf("\nWaiting customers = %d\n", waiting);

      sem_post(&customers);

      pthread_mutex_unlock(&mutex);

      sem_wait(&barber);

      printf("Customer %d got haircut\n", id);
   }
   else
   {
```

```
        pthread_mutex_unlock(&mutex);

        printf("\nCustomer %d left (No chair available)\n", id);
    }

    free(num);
}

int main()
{
    int n, i;

    printf("Enter number of chairs: ");
    scanf("%d", &chairs);

    printf("Enter number of customers: ");
    scanf("%d", &n);

    pthread_t barberThread, customerThread[n];

    sem_init(&customers, 0, 0);
    sem_init(&barber, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&barberThread, NULL, barber_func, NULL);

    for (i = 0; i < n; i++)
    {
        int *id = malloc(sizeof(int));
        *id = i + 1;

        pthread_create(&customerThread[i], NULL, customer_func, id);

        sleep(1);
    }

    for (i = 0; i < n; i++)
    {
        pthread_join(customerThread[i], NULL);
    }

    return 0;
}
```

10. Resource allocation

```
#include <stdio.h>

int main()
{
    int p, r, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &p);

    printf("Enter number of resources: ");
    scanf("%d", &r);

    int allocation[p][r];
    int request[p][r];
    int available[r];

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Request Matrix:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &request[i][j]);
        }
    }

    printf("\nEnter Available Resources:\n");
    for(i = 0; i < r; i++)
    {
        scanf("%d", &available[i]);
    }

    int finish[p];

    for(i = 0; i < p; i++)
```

```
    finish[i] = 0;

int count = 0;

while(count < p)
{
    int found = 0;

    for(i = 0; i < p; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j = 0; j < r; j++)
            {
                if(request[i][j] > available[j])
                {
                    possible = 0;
                    break;
                }
            }

            if(possible)
            {
                printf("\nProcess P%d executed", i);

                for(j = 0; j < r; j++)
                {
                    available[j] += allocation[i][j];
                }

                finish[i] = 1;
                found = 1;
                count++;
            }
        }
    }

    if(found == 0)
    {
        break;
    }
}

int deadlock = 0;
```

```
    for(i = 0; i < p; i++)
    {
        if(finish[i] == 0)
        {
            deadlock = 1;
            printf("\nProcess P%d is in Deadlock", i);
        }
    }

    if(deadlock == 0)
    {
        printf("\n\nNo Deadlock Detected\n");
    }

    return 0;
}
```

11. Bankers

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m], avail[m];

    printf("\nEnter Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter Max Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);

            // Input Validation: Allocation cannot be greater than Max
            if (alloc[i][j] > max[i][j]) {
```

```
            printf("\n[ERROR] Invalid Input! For Process P%d, Allocation (%d) cannot be
greater than Max (%d).\n", i, alloc[i][j], max[i][j]);
            return 0;
        }
    }
}

printf("\nEnter Available Resources:\n");
for (int i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}

// Calculating Need Matrix
// Formula: Need = Max - Allocation
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

// Arrays to keep track of finished processes and safe sequence
bool finish[n];
int safeSeq[n];
int work[m];

// Initializing tracking arrays
for (int i = 0; i < n; i++) finish[i] = false;
for (int i = 0; i < m; i++) work[i] = avail[i];

int count = 0;
while (count < n) {
    bool found = false;
    for (int p = 0; p < n; p++) {
        // Check if process is not finished
        if (!finish[p]) {
            // Check if Need <= Work (Available)
            bool canAllocate = true;
            for (int j = 0; j < m; j++) {
                if (need[p][j] > work[j]) {
                    canAllocate = false;
                    break;
                }
            }

            // If all resources can be allocated
            if (canAllocate) {
                // Free the allocated resources back to work/available
                for (int j = 0; j < m; j++) {
                    work[j] += alloc[p][j];
```

```
                }
                safeSeq[count++] = p;
                finish[p] = true;
                found = true;
            }
        }
    }

    // If no process could be executed in a full loop, system is in unsafe state
    if (!found) {
        printf("\nSystem is in UNSAFE STATE! (Deadlock detected or likely)\n");
        return 0;
    }
  }

  // Output the Safe Sequence
  printf("\nSystem is in SAFE STATE.\nSafe Sequence: ");
  for (int i = 0; i < n - 1; i++) {
      printf("P%d -> ", safeSeq[i]);
  }
  printf("P%d\n", safeSeq[n - 1]);

  return 0;
}
```

—----------

Step 1: Terminal open করো
Step 2: file create করো (vim sjf.c)
Step 3: Insert mode ● keyboard এ চাপো: 👉 i
Step 4: Code paste করো
Step 5: Save & exit (Esc :wq enter)
Step 6: Compile (gcc sjf.c -o sjf)
Step 7: Run (./sjf)